7

MÉTODOS DE MONTE CARLO (PARTE 2)

ALGORITMOS ON-POLICY VS OFF-POLICY

- Monte Carlo ES siempre comienza con un estado-acción aleatorio.

ALGORITMOS ON-POLICY VS OFF-POLICY

- ¿Cuándo usar Monte Carlo ES?

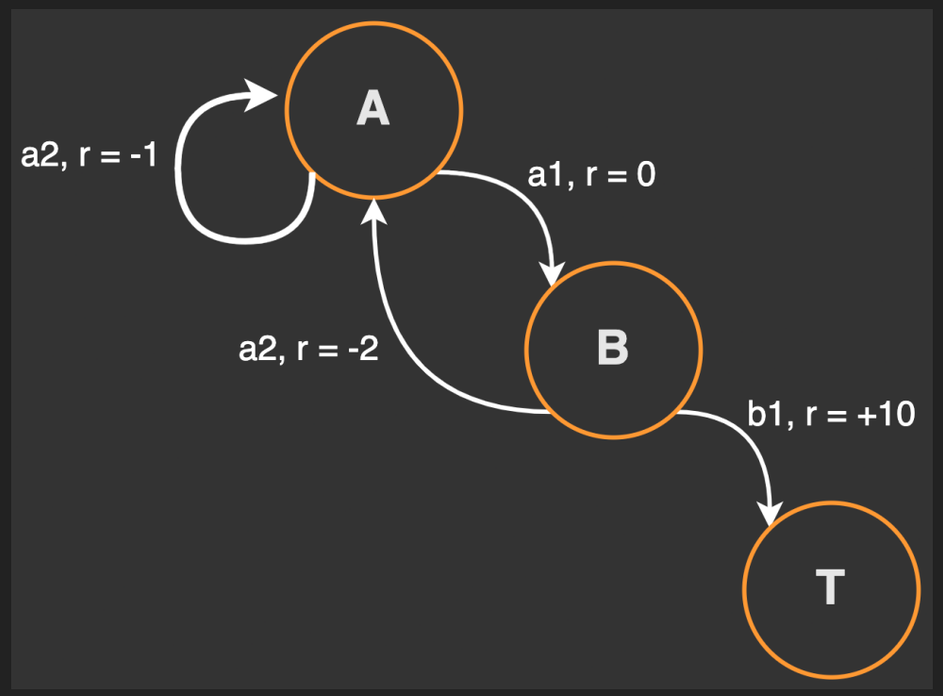
Entorno	¿Permite Exploring Starts?	Motivo
Péndulo invertido	Sí	Si se discretiza el espacio de estados (redondeo de la posición y velocidad de la barra)
Tic Tac Toe	Sí	Espacio pequeño, estados/acciones fácilmente controlables
Blackjack (simplificado)	Sí	Se puede forzar cualquier par estado-acción inicial
Gridworld (mundos de cuadrícula)	Sí	Se puede colocar al agente en cualquier celda con cualquier acción
Nim / Pig Dice Game	Sí	Estados discretos y manipulables
Racetrack (Sutton & Barto)	Sí	Estados discretizados y controlables
Connect Four (versión pequeña)	No	No se puede si una columna está llena.
Ajedrez, Go, Shogi	No	Estados complejos, no todos alcanzables segun las reglas de juego
Póker / Juegos con estados ocultos	No	No se conoce el estado completo, imposible establecer pares estado-acción arbitrarios
Dominó (simplificado)	No	No se puede iniciar una partida con cualquier ficha en cualquier lugar: las jugadas deben respetar el valor de conexión.

ALGORITMOS ON-POLICY VS OFF-POLICY

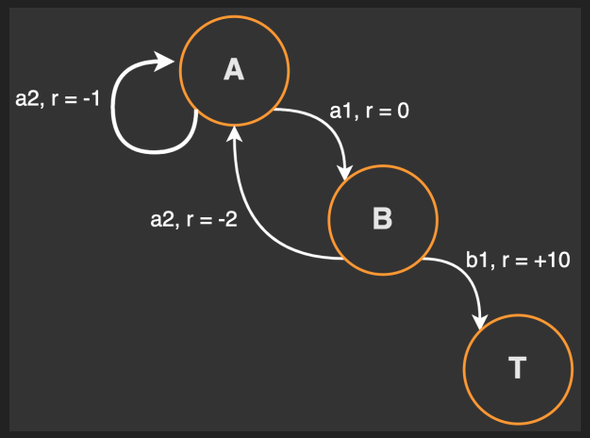
- ¿Qué sucede cuando no podemos garantizar que el agente comience en cada posible par estado-acción?
- Cuando esto ocurre se utilizan 2 métodos o enfoques:
  - ✓ On-policy (en la política)
  - ✓ Off-policy (fuera de la política)

EJEMPLO

- Política Objetivo (π - Target Policy): La que queremos evaluar.
  - ✓ Desde A: Siempre toma acción a1. (Probabilidad de a1 = 1.0, Probabilidad de a2 = 0.0)
  - ✓ Desde B: Siempre toma acción b1. (Probabilidad de b1 = 1.0, Probabilidad de b2 = 0.0)
    (Se busca estimar el valor esperado de empezar en A y seguir esta política π).
- Política de Comportamiento (b - Behavior Policy): La que usamos para explorar (aleatoria equiprobable).
  - ✓ Desde A: Toma acción a1 con probabilidad 0.5, acción a2 con probabilidad 0.5.
  - ✓ Desde B: Toma acción b1 con probabilidad 0.5, acción b2 con probabilidad 0.5.



EJEMPLO



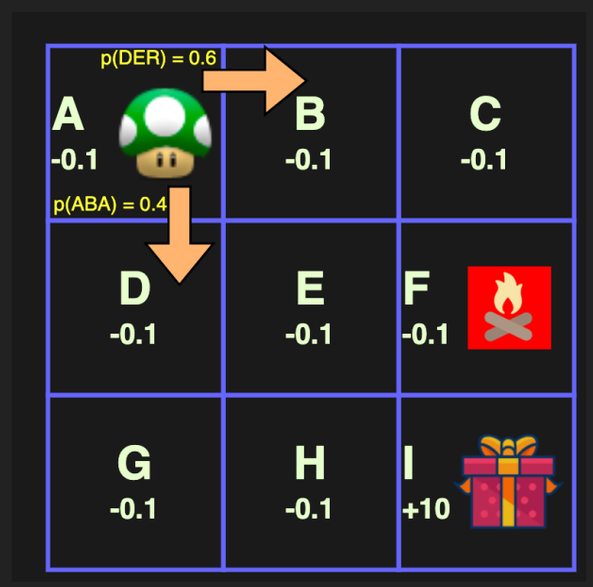
- Generando Datos con la Política de Comportamiento (b)
  - ✓ Episodio 1: A -> B -> T (Acciones: a1, b1). Retorno G desde A = 10.
  - ✓ Episodio 2: A -> A -> B -> T (Acciones: a2, a1, b1). Retorno G desde A = 9.
  - ✓ Episodio 3: A -> B -> A -> B -> T (Acciones: a1, b2, a1, b1). Retorno G desde A = 8.
- El problema es que los datos que tenemos no nos dicen directamente qué habría pasado si hubiéramos estado siguiendo la política objetivo.
- Observamos los retornos 10, 9 y 8. Si los promediamos (≈ 9), ¿refleja eso el valor de seguir la política objetivo π?
- No, porque los episodios 2 y 3 se generaron tomando acciones (como la acción a2 desde A en el episodio 2, o la acción b2 desde B en el episodio 3) que la política objetivo π nunca realizaría.
- La política π siempre tomaría a1 desde A y b1 desde B. Los resultados 9 y 8 están influenciados por la exploración aleatoria de la política de comportamiento b.

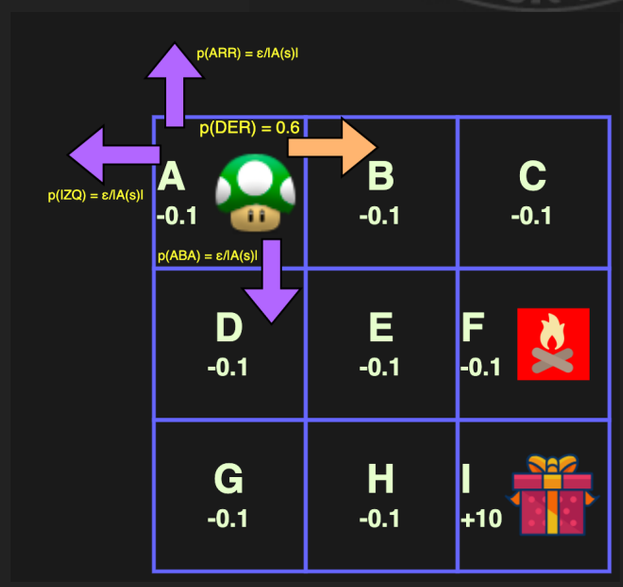
ALGORITMOS ON-POLICY VS OFF-POLICY

- Los métodos on-policy aprenden sobre la política que el agente está usando realmente (Ej. Monte Carlo ES).
- Los métodos off-policy pueden aprender sobre una política diferente incluso mientras el agente está haciendo otra cosa.
- Analogía: Si uno quisiera aprender manejar un coche tendría 2 opciones:
  - ✓ on-policy: Es como animarse y hacer. Aprendo sobre la marcha (autodidacta)
  - ✓ off-policy: Es observar a alguien más que ya domina el manejo del coche.

ALGORITMOS ON-POLICY VS OFF-POLICY

- Para resolver el problema de los comienzos exploratorios la técnica mas común es usar políticas suaves (soft policies).
- Soft policies -> en cada estado la política tiene al menos una pequeña probabilidad de elegir cualquiera de las acciones.
- Ya que no hay ES, la propia política de comportamiento debe incluir un componente de exploración.
- Cuando Soft policies estima que una acción es la mejor aun así prueba otras acciones de vez en cuando para asegurarse de que no se está perdiendo algo mejor.
- Una forma común de hacer esto es con algo llamado política épsilon-voraz (epsilon-greedy).





POLÍTICA EPSILON-GREEDY

- Con una probabilidad de 1-ε, el agente elige la acción que cree que es mejor. Pero luego, con una pequeña probabilidad ε, simplemente elige una acción aleatoria.
- ε-greedy en gran medida está usando su conocimiento, pero también se asegura de explorar un poco.
- En lugar de hacer la política completamente greedy, la hacemos épsilon-greedy.
- 1-ε+ε /|A(s)|, es la probabilidad de que la acción greedy sea seleccionada.

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(s)| & si \quad a = arg\max_a Q(s,a) \\ \varepsilon/|\mathcal{A}(s)| & si \quad a \neq arg\max_a Q(s,a) \end{cases}$$

GLIE

- Es el concepto de reducir gradualmente ε a 0.
- Permite que el agente:
  - ✓ Explore suficiente al principio (cuando ε es grande) para descubrir buenos valores Q.
  - ✓ Explote más el conocimiento adquirido posteriormente (cuando ε es pequeño) para converger hacia la política óptima.
- Se busca que ε tienda a 0 a medida que el tiempo/episodios tienden a infinito.
- Una forma de implementarlo es hacer $\varepsilon_k = 1 / k$, donde k es el número de episodio. A medida que k aumenta, ε_k se acerca a 0.
- GLIE significa "Greedy in the Limit with Infinite Exploration" (Codicioso en el Límite con Exploración Infinita).
- Garantiza que el algoritmo converja a la política óptima (π^*).

MÉTODO OFF-POLICY

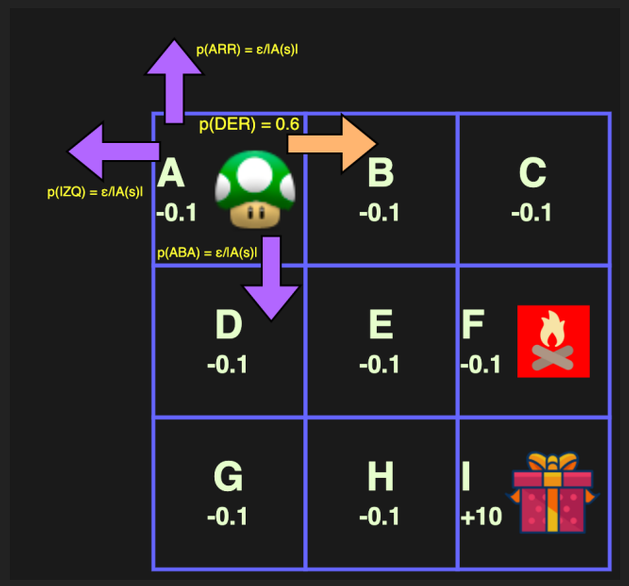
- ¿Por qué querríamos alguna vez aprender sobre una política mientras el agente se comporta realmente de acuerdo con una completamente diferente?
- Una ventaja en hacer eso es que se puede aprender sobre la política óptima, la que es realmente la mejor, mientras el agente se comporta de una manera que le permite explorar más.
- De modo que se puede estar probando acciones diferentes mientras se sigue aprendiendo sobre lo que es verdaderamente mejor.
- Con el aprendizaje off-policy, se puede aprender de datos que fueron generados por otra persona o incluso por un experto humano. No tiene porque hacer toda la exploración el agente mismo en todos los casos.

MÉTODO OFF-POLICY

- Política objetivo (target policy - π), la que se quiere aprender.
- Política de comportamiento (behavior policy - μ o b), es lo que el agente hace para explorar. Es la política que genera realmente los datos.

MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING – IS)

- El problema es que los G o Returns[] que tenemos no nos dicen directamente qué habría pasado si hubiéramos estado siguiendo la política objetivo.
- Para resolver esto se usa el muestreo por importancia (importance sampling). Es una forma de usar los datos que tenemos, aunque sean de una política diferente, para averiguar qué habría pasado si estuviéramos siguiendo la política que realmente nos interesa.
- IS es una técnica matemática que permite ponderar los resultados (los retornos G) obtenidos bajo la política de comportamiento (μ) para que reflejen mejor lo que habría sucedido bajo la política objetivo (π*).



MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING – IS)

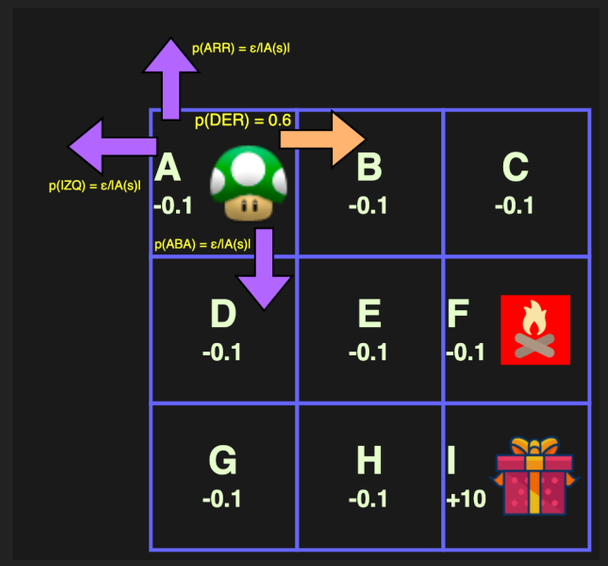
- Muestreo por importancia ordinario

$$\rho = \prod_{t=0}^{T-1} \frac{\pi(a_t|s_t)}{b(a_t|s_t)} \qquad V(s) \leftarrow \text{promedio de } \rho \cdot G$$

- Muestreo por importancia ponderado.

$$\rho = \prod_{t=0}^{T-1} \frac{\pi(a_t|s_t)}{b(a_t|s_t)} \qquad V(s) \leftarrow \frac{\sum \rho \cdot G}{\sum \rho}$$

- Donde G es el retorno total desde s_t



MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING – IS)

- Muestreo por importancia ordinario (pseudocódigo)

Para cada episodio:

Generar episodio: (s0, a0, r1, s1, a1, ..., sT)

G ← retorno total del episodio

ρ ← 1

Para t desde 0 hasta T−1:

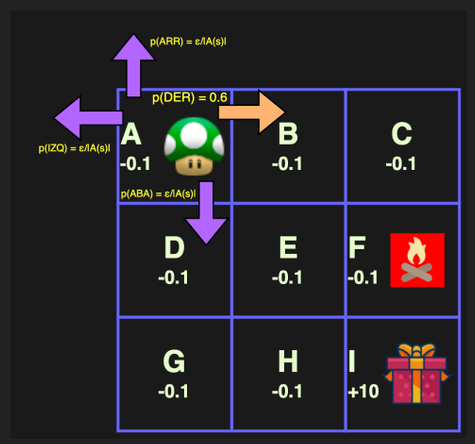
ρ ← ρ * π(at | st) / b(at | st)

V(s0) ← V(s0) + ρ * G

N(s0) ← N(s0) + 1

Para cada s:

V(s) ← V(s) / N(s)



MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING – IS)

- Muestreo por importancia ponderado. (pseudocódigo)

Para cada episodio:

Generar episodio: (s0, a0, r1, s1, a1, ..., sT)

G ← retorno total del episodio

ρ ← 1

Para t desde 0 hasta T−1:

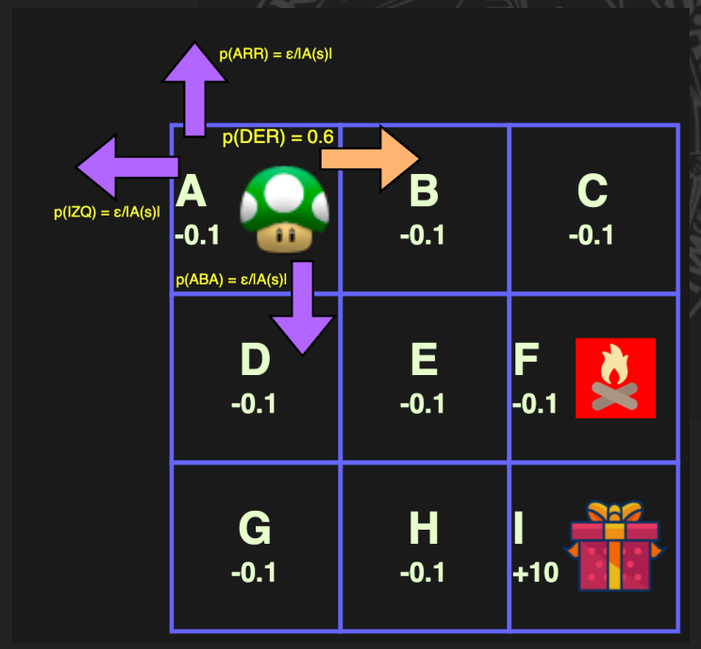
ρ ← ρ * π(at | st) / b(at | st)

W(s0) ← W(s0) + ρ * G

C(s0) ← C(s0) + ρ

Para cada s:

V(s) ← W(s) / C(s)



REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.